

LAB

Adjust the IBM Granite model parameters to refine the output

Contents

Introduction	2
Software requirements	2
Objective	2
Lab steps	2
Estimated duration to complete	2
Scenario	3
Background information	3
Challenge	3
Solution	3
Step 1: Confirm software setup	4
Overview	4
Step 2: Test initial classification output with default values for prompt parameters	5
Overview	5
Instructions	5
Step 3: Adjust a single parameter to improve the classification output.....	11
Overview	11
Instructions	11
Step 4: Adjust multiple parameters to refine the classification output further	13
Overview	13
Instructions	13
Step 5: Test initial summarization output with default values for parameters	19
Overview	19
Instructions	19
Step 6: Adjust a single parameter to improve the summarization output	24
Overview	24
Instructions	24
Step 7: Adjust multiple parameters to improve the summarization output	26
Overview	26
Instructions	26
Conclusion	29

Introduction

In this lab, you will refine IBM Granite model output by adjusting key parameters to improve performance for classification and summarization tasks. Building on the automated workflows implemented in the previous lab, you will now explore how tuning specific model parameters for prompting, such as `max_tokens`, `top_k`, `top_p`, `repetition_penalty`, and `stopping_sequence`, impact the quality of results.

These refinements will help you achieve consistent, actionable output tailored to specific stakeholder requirements.

Software requirements

To complete this lab, you must have access to the following:

- **Replicate account:** Create a Replicate account and obtain an API token.
- **Google Colab:** Create a Google account to use Colab for running the notebook.
- **Python packages:** Install required packages (such as `langchain_community`, `os`, `replicate`, `google.colab`).

Objective

After completing this lab, you should be able to:

- Adjust the IBM Granite model parameters for prompting to refine the output

Lab steps

This lab requires you to complete the following steps:

- Step 1: Confirm software setup
- Step 2: Test initial classification output with default values for prompt parameters
- Step 3: Adjust a single parameter to improve the classification output
- Step 4: Adjust multiple parameters to refine the classification output further
- Step 5: Test initial summarization output with default values for parameters
- Step 6: Adjust a single parameter to improve the summarization output
- Step 7: Adjust multiple parameters to improve the summarization output

Estimated duration to complete

30 minutes

Scenario

Background information

As a product manager at SmartTechX, a mid-sized smartphone company, your responsibilities include analyzing customer reviews to identify sentiment trends and tagging focus areas and summarizing cross-functional team meetings to provide stakeholders with actionable insights.

In the previous lab, you successfully automated these workflows using the **granite-3.3-8b-instruct** model. However, your stakeholders have highlighted issues with the consistency and precision of the output. Some classification results lack focus area tagging, while meeting summaries occasionally miss key details or fail to provide a clear structure. To address these challenges, you'll refine the model's output by adjusting key parameters, ensuring they align with stakeholder expectations.

Challenge

Despite the success of automation, the stakeholders raised several issues with the model's output:

- **Classification task:** Some output includes redundant or inconsistent phrases. The output also suffers from incomplete tagging of focus areas, impacting usability.
- **Summarization task:** Some summaries are overly generic and fail to capture key decisions and action items. Also, the structure of output needs to align better with stakeholder expectations.

Solution

You believe you can refine both classification and summarization output by adjusting the parameters for **granite-3.3-8b-instruct** model in the following ways:

- Use **max_tokens** to control output length, ensuring concise and focused responses.
- Experiment with parameters, such as **top_k**, **top_p**, **repetition_penalty**, and **stopping_sequence**, to improve the coherence, relevance, and structure of output.
- Evaluate the impact of each adjustment and iterate as needed to align results with stakeholder needs.

By adjusting these parameters, you hope to refine the output for both classification and summarization tasks, improving the efficiency, accuracy, and decision making across departments.

Step 1: Confirm software setup

Overview

In this step, ensure that you have completed the following steps from lab 1:

- Step 1: Create a GitHub account
- Step 2: Set up your environment and initialize the IBM Granite model
- Step 3: Sign up for Google Colab

If you have not completed these steps, refer to the “Lab 1: Prompt an IBM Granite model to classify and summarize data” document for details and complete the steps now.

Step 2: Test initial classification output with default values for prompt parameters**Overview**

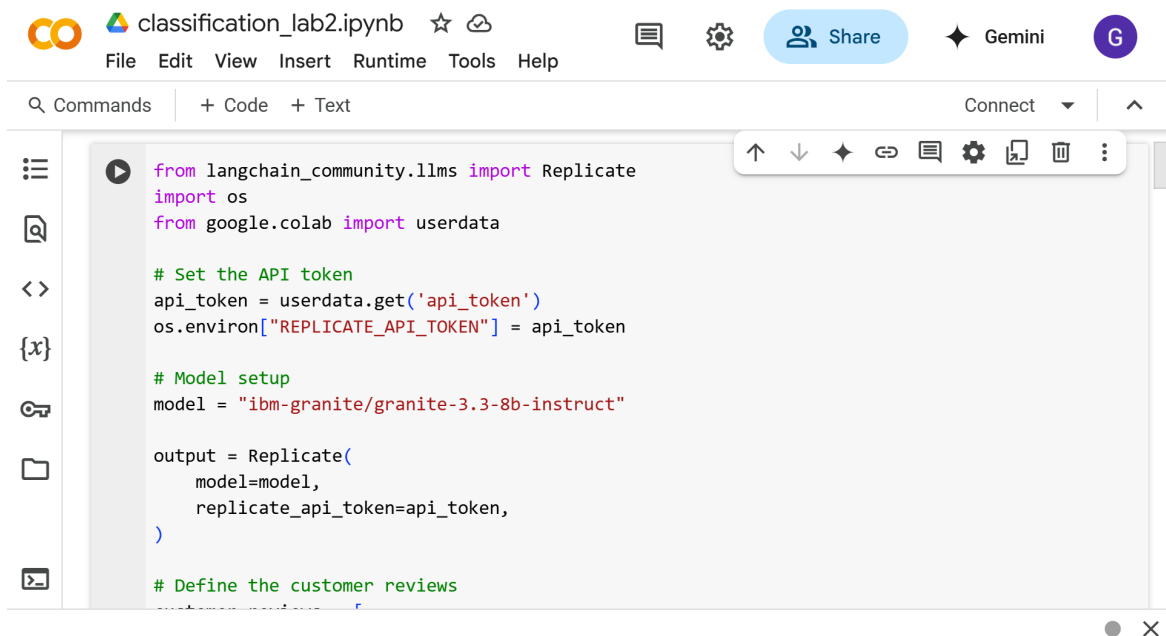
In this step, you will input customer reviews into the model, run a basic prompt with the default multiple model parameters to classify the reviews, and analyze the output to identify areas for improvement.

Instructions

1. Input sample customer reviews to test the model. Type these customer reviews in the **Instruction** field:

```
# Define the customer reviews
customer_reviews = [
    "The battery lasts all day, but the phone gets hot during gaming.",
    "The screen is too dim outdoors, but I love the colors indoors.",
    "This phone is fast, but it keeps crashing when I open certain apps."
]

# Refine the prompt to include reviews
reviews_text = "\n".join([f"Review {i+1}: {review}" for i, review in enumerate(customer_reviews)])
```



The screenshot shows a Jupyter Notebook titled 'classification_lab2.ipynb'. The interface includes a top bar with the Colab logo, file name, and icons for saving, sharing, and connecting to Gemini. Below the top bar is a menu bar with options: File, Edit, View, Insert, Runtime, Tools, and Help. The main area is divided into a left sidebar with icons for file explorer, search, and other functions, and a central code editor. The code editor contains the following Python code:

```
from langchain_community.llms import Replicate
import os
from google.colab import userdata

# Set the API token
api_token = userdata.get('api_token')
os.environ["REPLICATE_API_TOKEN"] = api_token

# Model setup
model = "ibm-granite/granite-3.3-8b-instruct"

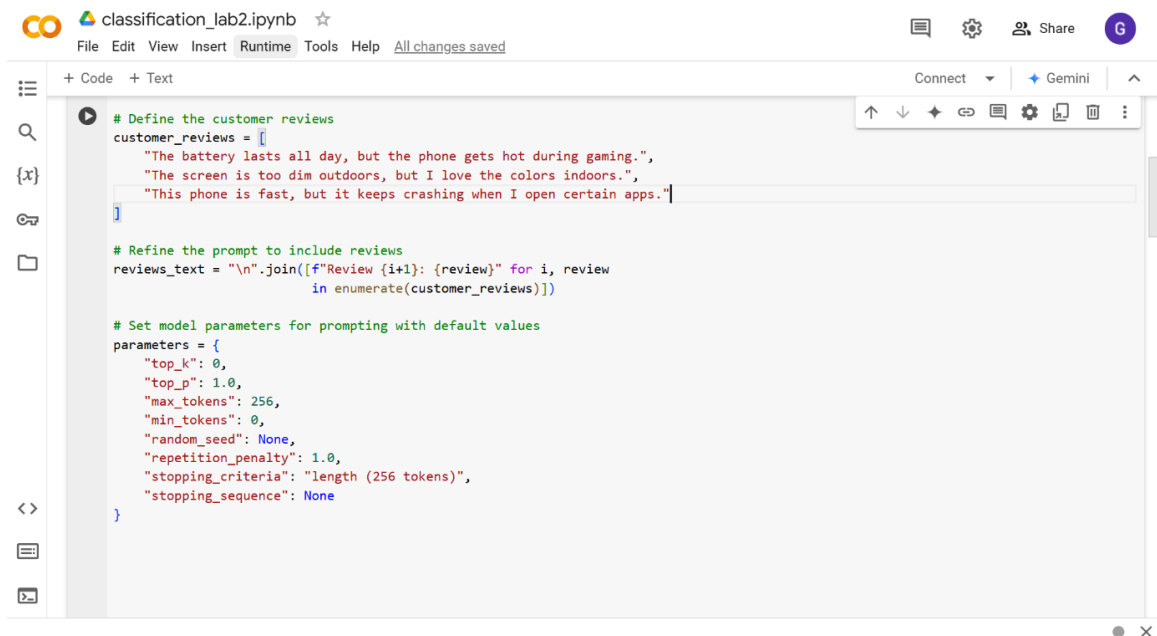
output = Replicate(
    model=model,
    replicate_api_token=api_token,
)

# Define the customer reviews
```

2. **Remember:** Adjusting parameters that control token selection, length, or diversity can significantly refine the quality of the responses. These include:
- **top_k:** Limits the number of possible next tokens considered when generating text, selecting from the "top k" most likely options based on their probability. This focuses the output and reduces the likelihood of unexpected or irrelevant words, acting as a filter for the most probable next words.
 - **top_p:** Influences diversity by restricting token selection to the most probable options within a cumulative probability threshold. It ensures that the model chooses from a more focused set of options, improving response coherence while maintaining diversity.
 - **max_tokens:** Controls the maximum length of the model's response. By setting a token limit, this parameter prevents overly lengthy responses while helping to focus the model's output within a manageable size.
 - **min_tokens:** Sets a minimum length for the output, ensuring the model generates sufficiently detailed responses, which is useful for tasks requiring comprehensive explanations or summaries
 - **random_seed:** Ensures reproducibility by fixing the randomness in model output
 - **repetition_penalty:** Reduces redundancy by penalizing repeated tokens in the output. This encourages the model to generate more diverse and varied language, preventing it from repeating the same words or phrases unnecessarily.
 - **stopping_criteria:** Defines the conditions under which the model stops generating tokens
 - **stopping_sequence:** Halts the response when the model encounters a predefined string, such as a newline character or specific word

Input the default values for parameters. Type this code in the **Instruction** field:

```
# Set model parameters for prompting with default values
parameters = {
    "top_k": 0,
    "top_p": 1.0,
    "max_tokens": 256,
    "min_tokens": 0,
    "random_seed": None,
    "repetition_penalty": 1.0,
    "stopping_criteria": "length (256 tokens)",
    "stopping_sequence": None
}
```

```

classification_lab2.ipynb
File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text

# Define the customer reviews
customer_reviews = [
    "The battery lasts all day, but the phone gets hot during gaming.",
    "The screen is too dim outdoors, but I love the colors indoors.",
    "This phone is fast, but it keeps crashing when I open certain apps."
]

# Refine the prompt to include reviews
reviews_text = "\n".join([f"Review {i+1}: {review}" for i, review
                           in enumerate(customer_reviews)])

# Set model parameters for prompting with default values
parameters = {
    "top_k": 0,
    "top_p": 1.0,
    "max_tokens": 256,
    "min_tokens": 0,
    "random_seed": None,
    "repetition_penalty": 1.0,
    "stopping_criteria": "length (256 tokens)",
    "stopping_sequence": None
}

```

- Input the initial prompt. Type this code in the **Instruction** field:

```

# Add initial prompt
refined_prompt = f"""
Classify these reviews as positive, negative, or mixed, and tag
relevant focus areas such as battery life, screen quality, or
performance

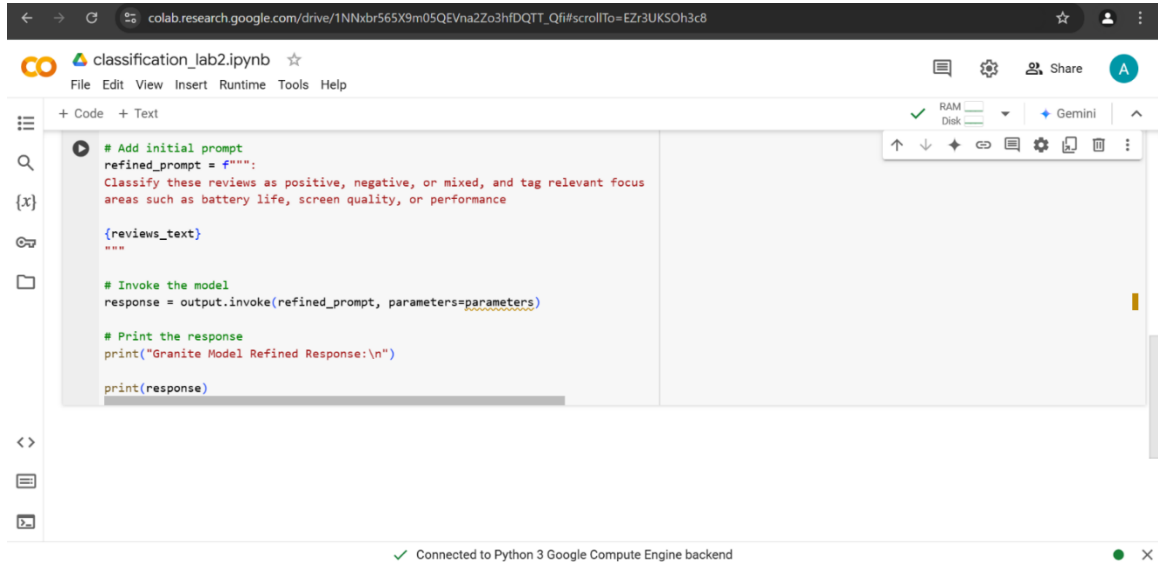
{reviews_text}
"""

# Invoke the model
response = output.invoke(refined_prompt, parameters=parameters)

# Print the response
print("Granite Model Refined Response:\n")

print(response)

```



The screenshot shows a Google Colab notebook interface. The notebook is titled "classification_lab2.ipynb" and is connected to a Python 3 Google Compute Engine backend. The code in the notebook is as follows:

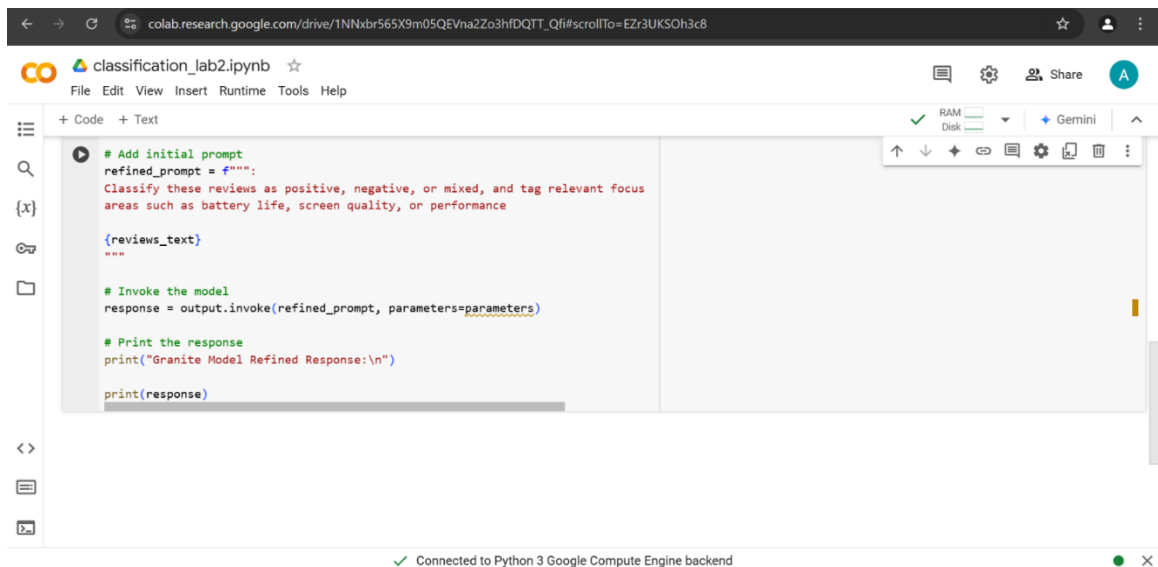
```
# Add initial prompt
refined_prompt = f"""
Classify these reviews as positive, negative, or mixed, and tag relevant focus
areas such as battery life, screen quality, or performance

{reviews_text}
"""

# Invoke the model
response = output.invoke(refined_prompt, parameters=parameters)

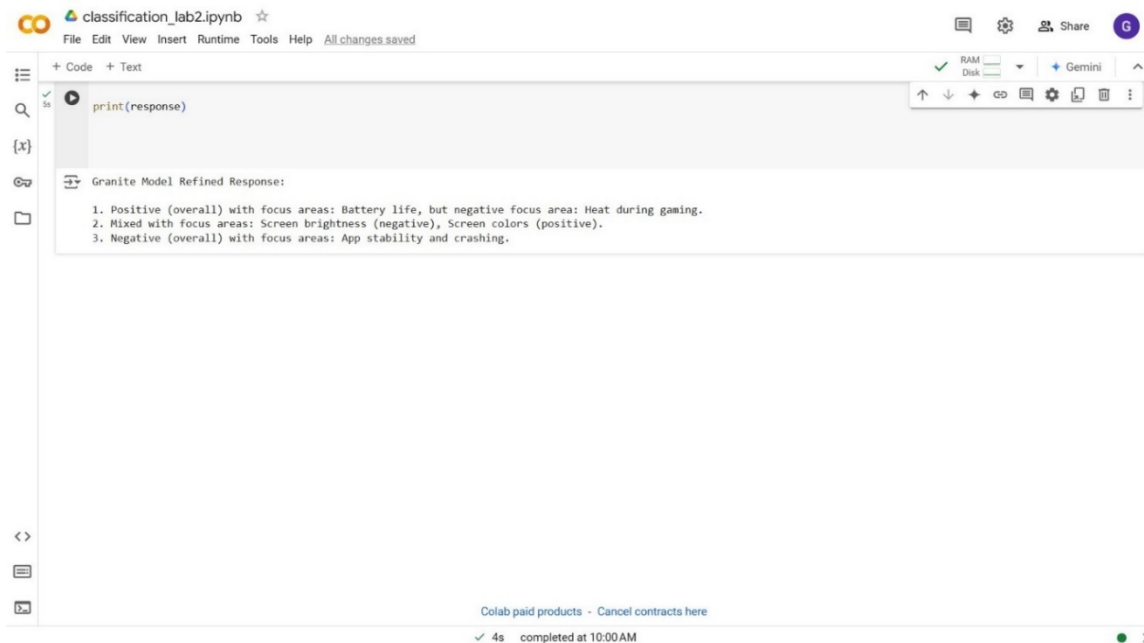
# Print the response
print("Granite Model Refined Response:\n")
print(response)
```

4. Select the **Play** button to run the initial prompt to test the default values for parameters.



This screenshot is identical to the one above, showing the same Google Colab notebook interface with the same Python code for model refinement.

5. Read the output that the model generates. Note that the output highlights the focus areas effectively, addressing positive and negative aspects. However, the response is not concise, making it less suitable for quick evaluation. To address this, you need to refine the prompt by adjusting the top_k parameter to ensure more granular and concise sentiment categorization.



The screenshot shows a Jupyter Notebook titled 'classification_lab2.ipynb'. The code cell contains the line `print(response)`. The output of the cell is displayed below the code, showing the 'Granite Model Refined Response' as a numbered list of three items:

```
Granite Model Refined Response:
```

1. Positive (overall) with focus areas: Battery life, but negative focus area: Heat during gaming.
2. Mixed with focus areas: Screen brightness (negative), Screen colors (positive).
3. Negative (overall) with focus areas: App stability and crashing.

The bottom of the notebook interface shows a status bar indicating 'Colab paid products - Cancel contracts here' and a completion time of '4s completed at 10:00 AM'.

Step 3: Adjust a single parameter to improve the classification output

Overview

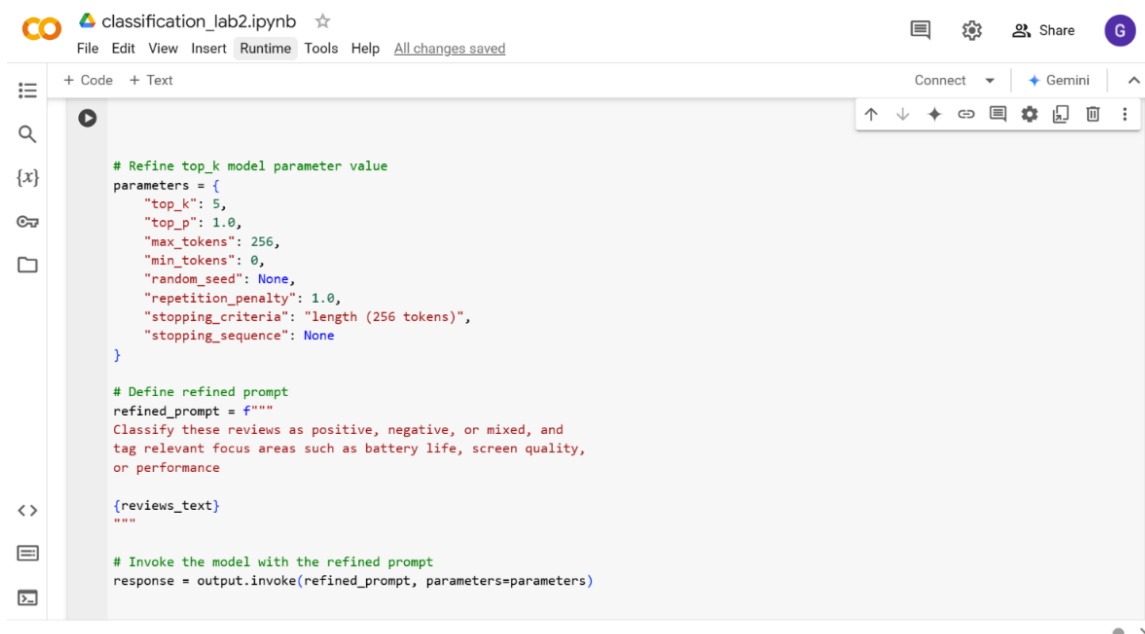
In this step, you will adjust the `top_k` parameter to add granularity by explicitly categorizing each sentiment in a review.

Instructions

1. **Remember:** Adjusting the `top_k` parameter helps the model focus on the most likely options, improving precision in its classifications. By limiting the `top_k` value to 5, the model considers only the top 5 most probable tokens, reducing ambiguity and enhancing the accuracy of the output.

Update the `top_k` parameter by setting it to 5. Type this code in the **Instruction** field:

```
# Refine top_k model parameter value
parameters = {
    "top_k": 5,
    "top_p": 1.0,
    "max_tokens": 256,
    "min_tokens": 0,
    "random_seed": None,
    "repetition_penalty": 1.0,
    "stopping_criteria": "length (256 tokens)",
    "stopping_sequence": None
}
```



The screenshot shows a Jupyter Notebook titled 'classification_lab2.ipynb'. The interface includes a top bar with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', 'Help', and 'All changes saved'. Below this is a toolbar with icons for code, text, and other functions. The main area displays the following code:

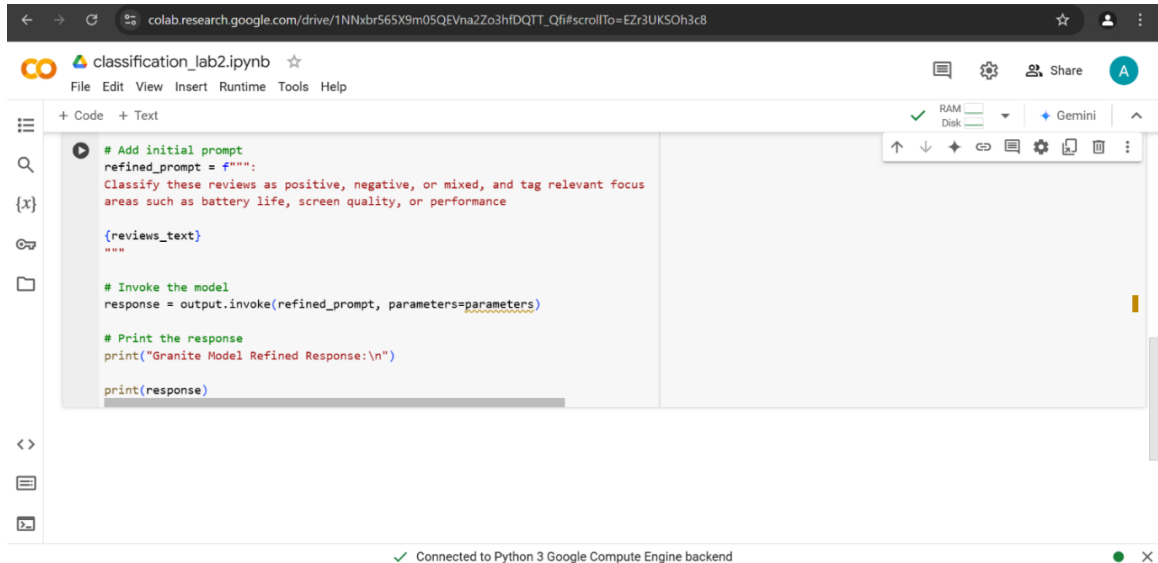
```
# Refine top_k model parameter value
parameters = {
    "top_k": 5,
    "top_p": 1.0,
    "max_tokens": 256,
    "min_tokens": 0,
    "random_seed": None,
    "repetition_penalty": 1.0,
    "stopping_criteria": "length (256 tokens)",
    "stopping_sequence": None
}

# Define refined prompt
refined_prompt = f"""
Classify these reviews as positive, negative, or mixed, and
tag relevant focus areas such as battery life, screen quality,
or performance

{reviews_text}
"""

# Invoke the model with the refined prompt
response = output.invoke(refined_prompt, parameters=parameters)
```

2. Select the **Play** button to run the prompt.



The screenshot shows a Google Colab notebook titled 'classification_lab2.ipynb'. The code in the first cell is as follows:

```
# Add initial prompt
refined_prompt = f"""
Classify these reviews as positive, negative, or mixed, and tag relevant focus
areas such as battery life, screen quality, or performance

{reviews_text}
"""

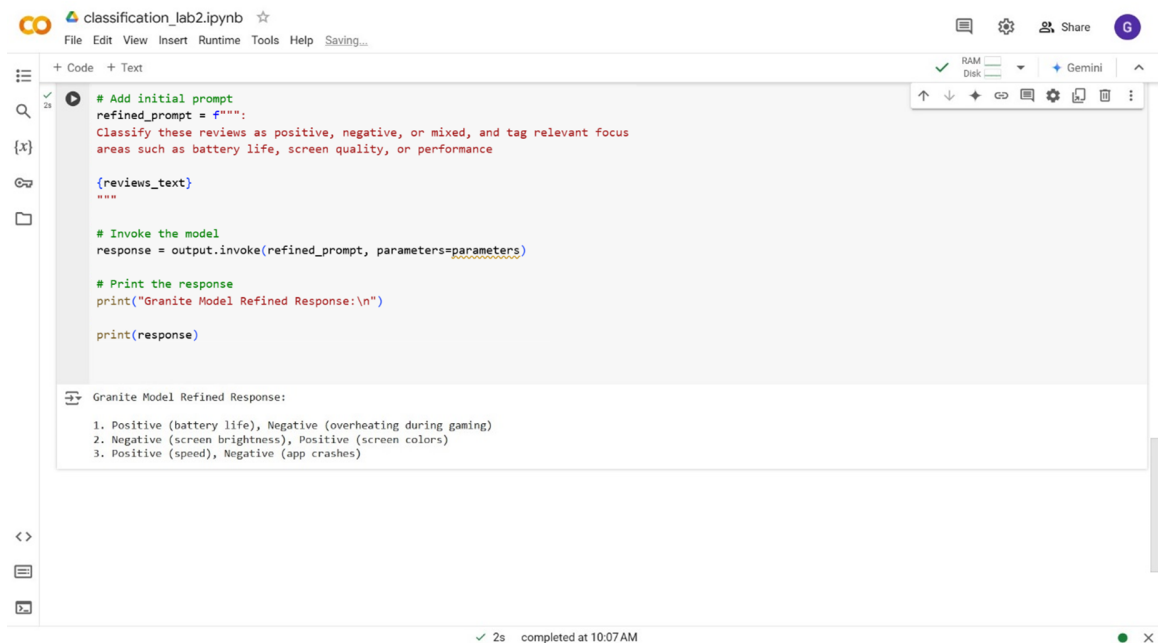
# Invoke the model
response = output.invoke(refined_prompt, parameters=parameters)

# Print the response
print("Granite Model Refined Response:\n")
print(response)
```

The notebook interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help), a toolbar with icons for code, text, and execution, and a status bar at the bottom indicating 'Connected to Python 3 Google Compute Engine backend'.

3. Read the output that the model generates. Note that the output effectively categorizes sentiments but could be more concise. While `top_k = 0` focused on broad categories of sentiments and provided high-level classification, adjusting the parameter to `top_k = 5` added granularities by explicitly categorizing each sentiment in a review.

To further improve the response, consider adjusting multiple parameters in the next step to refine the coherence, precision, and relevance of the classification output.



The screenshot shows the same Google Colab notebook, but now the output of the model is visible. The code is identical to the previous screenshot. The output is as follows:

```
Granite Model Refined Response:
1. Positive (battery life), Negative (overheating during gaming)
2. Negative (screen brightness), Positive (screen colors)
3. Positive (speed), Negative (app crashes)
```

The notebook interface is the same, but the status bar at the bottom now shows '2s completed at 10:07 AM'.

Step 4: Adjust multiple parameters to refine the classification output further

Overview

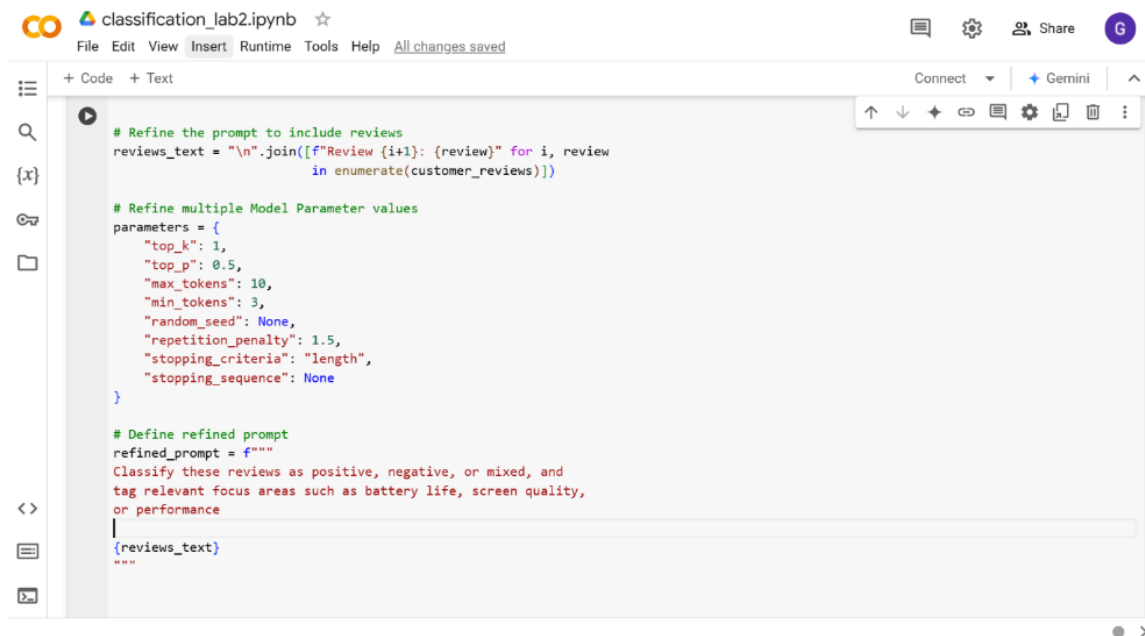
In this step, you will adjust multiple parameters to refine coherence, precision, and relevance in the classification results.

Instructions

1. Adjust the following model parameters to keep the output short and concise:
 - **top_k**: Adjusting the value to 1 to restrict the response to the most likely token.
 - **top_p**: Adjusting the value to 0.5 to further filter responses for coherence and relevance
 - **max_tokens**: Adjusting the value to 10 to ensure concise output.
 - **repetition_penalty**: Increasing the repetition penalty to 1.5 to avoid redundant phrasing.

Input the refined multiple parameter values. Type this code in the **Instruction** field:

```
# Refine multiple Model Parameter values
parameters = {
    "top_k": 1,
    "top_p": 0.5,
    "max_tokens": 10,
    "min_tokens": 3,
    "random_seed": None,
    "repetition_penalty": 1.5,
    "stopping_criteria": "length",
    "stopping_sequence": None
}
```



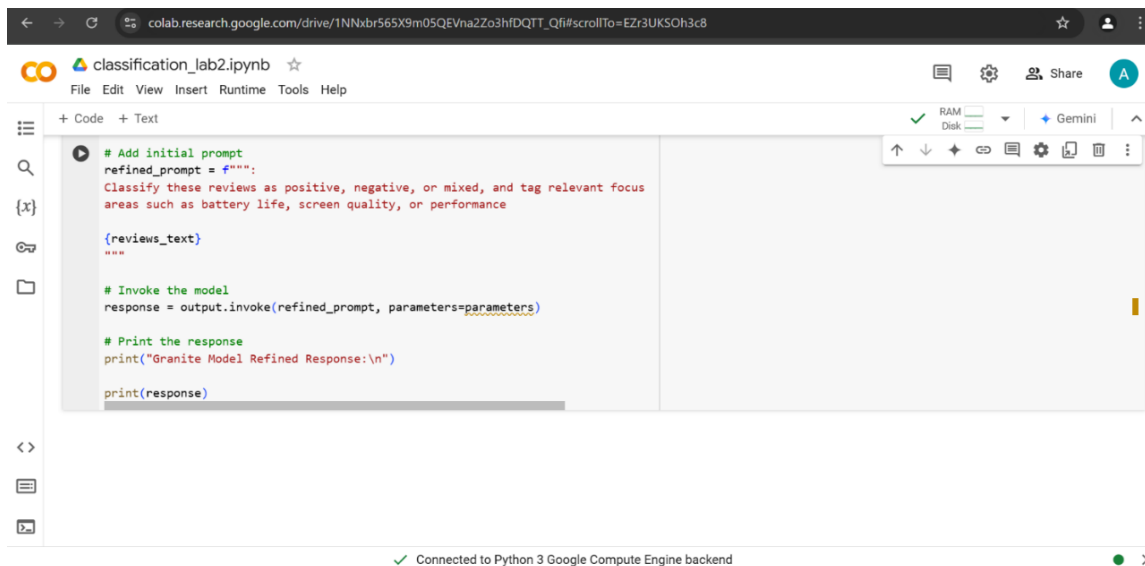
The screenshot shows a Jupyter Notebook titled 'classification_lab2.ipynb'. The code defines a function to refine a prompt by including reviews and setting model parameters. The refined prompt is then defined.

```
# Refine the prompt to include reviews
reviews_text = "\n".join([f"Review {i+1}: {review}" for i, review
                           in enumerate(customer_reviews)])

# Refine multiple Model Parameter values
parameters = {
    "top_k": 1,
    "top_p": 0.5,
    "max_tokens": 10,
    "min_tokens": 3,
    "random_seed": None,
    "repetition_penalty": 1.5,
    "stopping_criteria": "length",
    "stopping_sequence": None
}

# Define refined prompt
refined_prompt = f"""
Classify these reviews as positive, negative, or mixed, and
tag relevant focus areas such as battery life, screen quality,
or performance
{reviews_text}
"""
```

2. Select the **Play** button to run the prompt.



The screenshot shows the same Jupyter Notebook with additional code to invoke the model and print the response. The code is as follows:

```
# Add initial prompt
refined_prompt = f"""
Classify these reviews as positive, negative, or mixed, and tag relevant focus
areas such as battery life, screen quality, or performance
{reviews_text}
"""

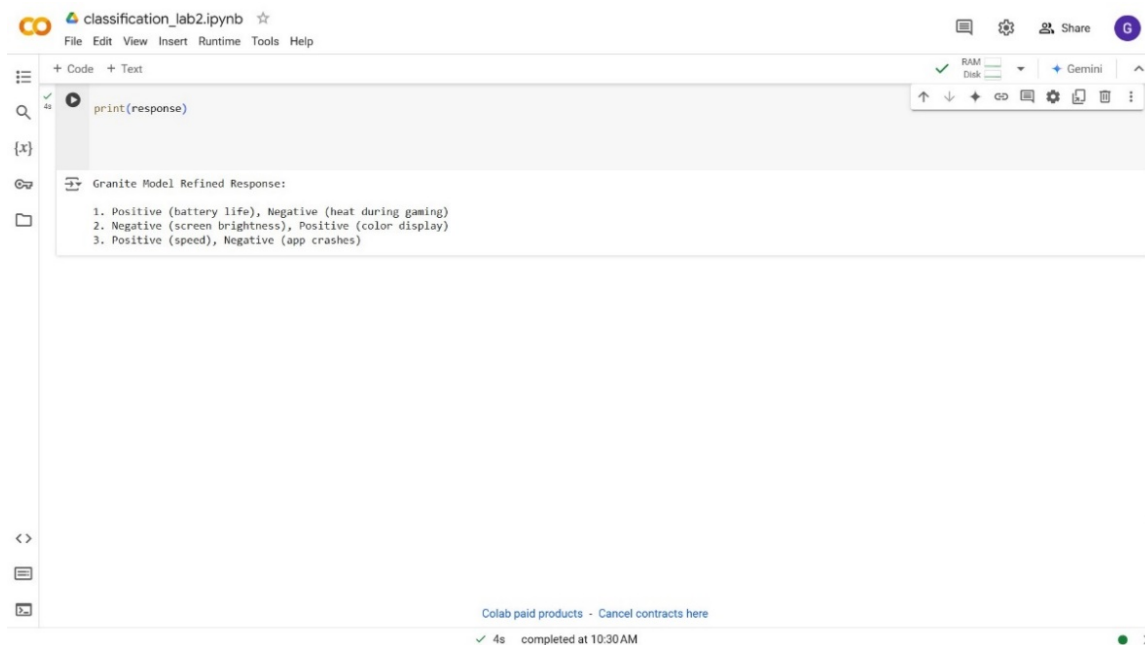
# Invoke the model
response = output.invoke(refined_prompt, parameters=parameters)

# Print the response
print("Granite Model Refined Response:\n")
print(response)
```

At the bottom of the notebook, a status bar indicates "Connected to Python 3 Google Compute Engine backend".

3. Read the output that the model generates. Note that the response effectively categorizes sentiments but could still benefit from further conciseness. Adjusting `max_tokens = 10` ensured concise output, while `repetition_penalty = 1.5` avoided redundant phrasing. `top_k = 1` restricted the response to the most likely token, while `top_p = 0.5` further filtered responses for coherence and relevance. However, there's still some scope to make the output more concise.

Now that you have refined the initial parameters, adjust the parameters again to make the output even more concise.



The screenshot shows a Jupyter Notebook titled 'classification_lab2.ipynb'. The code cell contains the line `print(response)`. The output cell displays the 'Granite Model Refined Response' as a numbered list:

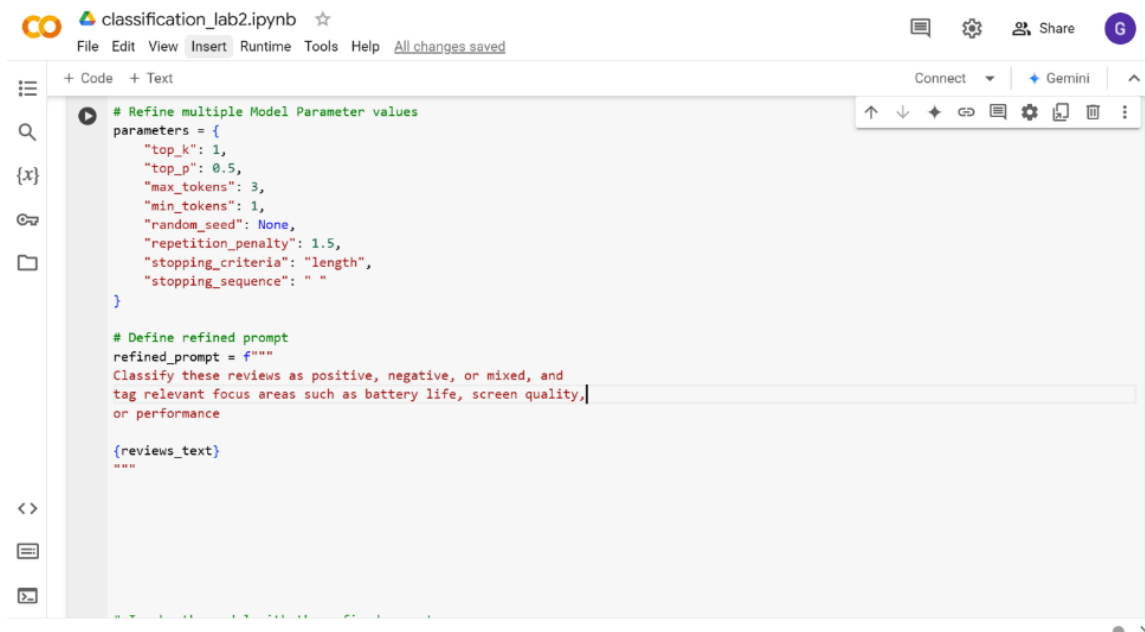
```
Granite Model Refined Response:
1. Positive (battery life), Negative (heat during gaming)
2. Negative (screen brightness), Positive (color display)
3. Positive (speed), Negative (app crashes)
```

The interface includes a left sidebar with icons for file explorer, search, and other notebook functions. The top bar shows 'File Edit View Insert Runtime Tools Help' and a 'Share' button. The bottom status bar indicates 'Colab paid products - Cancel contracts here' and '4s completed at 10:30AM'.

4. Adjust the following parameters again to make the output even more concise:
 - **max_tokens:** Reduce further to 3 to enforce brevity.
 - **top_k:** Maintain at 1 to select the most probable token for deterministic output.
 - **top_p:** Maintain at 0.5 to focus on high-probability tokens.
 - **min_tokens:** Set to 1 to ensure a minimum length for the output, ensuring the model generates detailed responses.
 - **random_seed:** Set to **None** for varied output or a fixed value for reproducibility.
 - **repetition_penalty:** Set to 1.5 to penalize repeated tokens and promote variety in the response.
 - **stopping_criteria:** Set to "length" to stop the model after a specified token count is reached.
 - **stopping_sequence:** Set to a space (" ") to force termination at natural breaks, such as word boundaries.

Input the refined multiple parameter values. Type this code in the **Instruction** field:

```
#Refine multiple Model Parameter values
parameters = {
    "top_k": 1,
    "top_p": 0.5,
    "max_tokens": 3,
    "min_tokens": 1,
    "random_seed": None,
    "repetition_penalty": 1.5,
    "stopping_criteria": "length",
    "stopping_sequence": " "
}
```



The screenshot shows a Jupyter Notebook titled 'classification_lab2.ipynb'. The code in the cell is as follows:

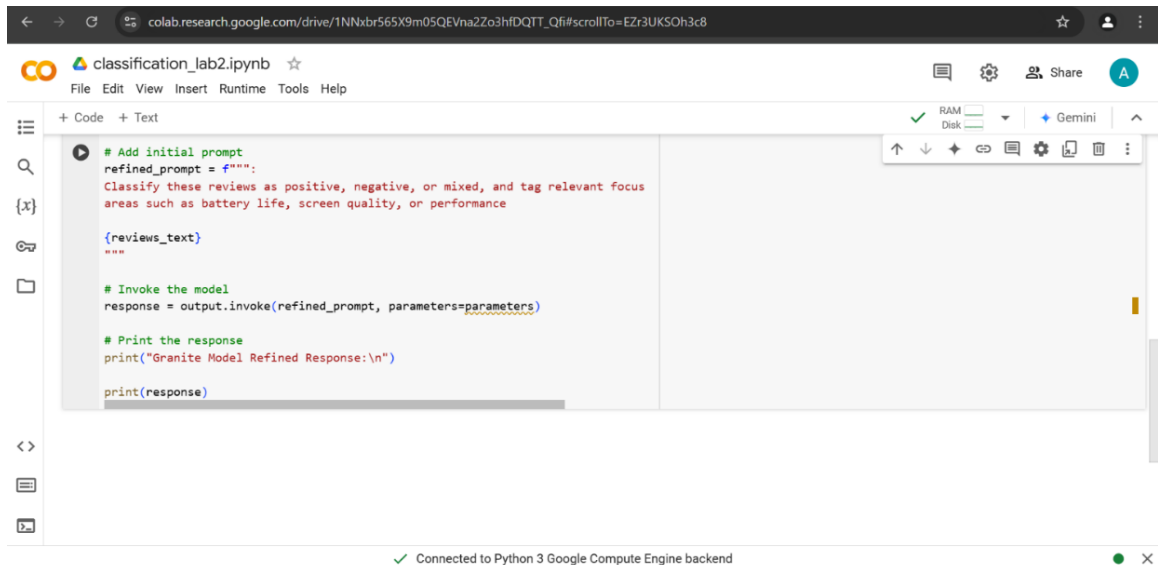
```
# Refine multiple Model Parameter values
parameters = {
    "top_k": 1,
    "top_p": 0.5,
    "max_tokens": 3,
    "min_tokens": 1,
    "random_seed": None,
    "repetition_penalty": 1.5,
    "stopping_criteria": "length",
    "stopping_sequence": " "
}

# Define refined prompt
refined_prompt = f"""
Classify these reviews as positive, negative, or mixed, and
tag relevant focus areas such as battery life, screen quality,
or performance

{reviews_text}
"""
```

The notebook interface includes a left sidebar with icons for file explorer, search, and other functions. The top bar shows 'File Edit View Insert Runtime Tools Help' and 'All changes saved'. The right sidebar has 'Connect' and 'Gemini' options.

5. Select the **Play** button to run the prompt.



The screenshot shows a Google Colab notebook titled 'classification_lab2.ipynb'. The code in the cell is as follows:

```
# Add initial prompt
refined_prompt = f"""
Classify these reviews as positive, negative, or mixed, and tag relevant focus
areas such as battery life, screen quality, or performance

{reviews_text}
"""

# Invoke the model
response = output.invoke(refined_prompt, parameters=parameters)

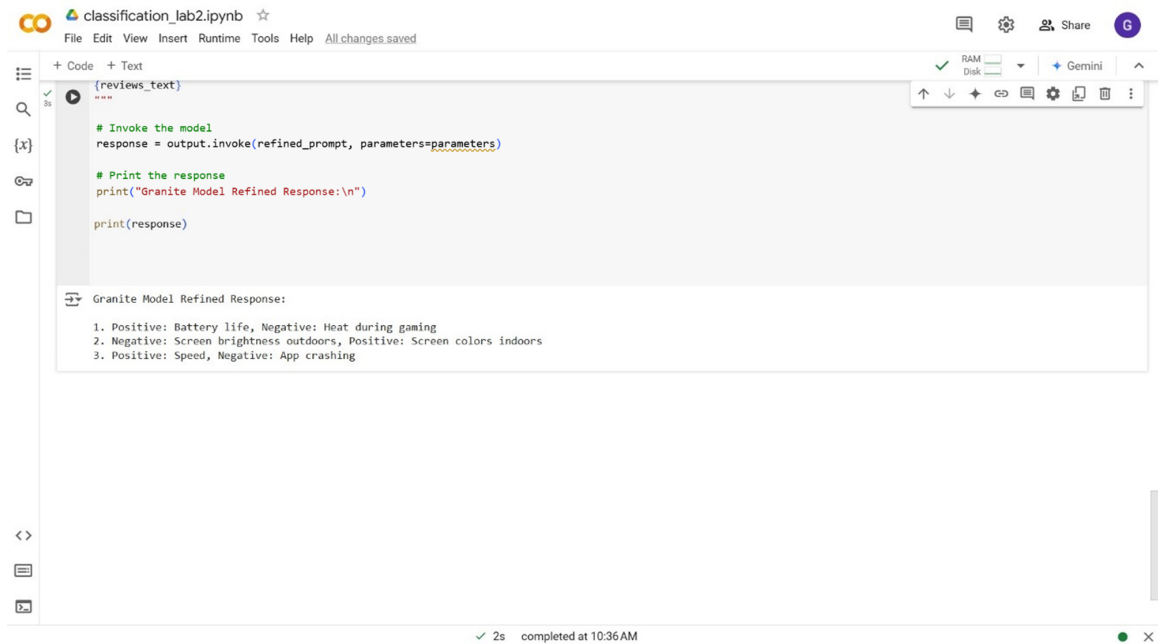
# Print the response
print("Granite Model Refined Response:\n")
print(response)
```

The notebook interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help), a toolbar with icons for code, text, and execution, and a status bar at the bottom indicating 'Connected to Python 3 Google Compute Engine backend'.

6. Read the output that the model generates. Note that no change in the output occurred, even after using very low token values and setting the sequence to stop upon encountering a single space character.

Remember: The model parameter values have a threshold beyond which any change in value would not bring any effect in the output. The models are trained to respond in a certain structure, in which case both model parameter changes and prompt modifications influence the output response.

You've completed the parameter adjustments for concise output. Next, let's test the initial summarization output with default values for parameters by inputting the meeting transcript.



The screenshot shows a Jupyter Notebook titled 'classification_lab2.ipynb'. The code cell contains the following Python code:

```
{reviews_text}  
.....  
  
# Invoke the model  
response = output.invoke(refined_prompt, parameters=parameters)  
  
# Print the response  
print("Granite Model Refined Response:\n")  
  
print(response)
```

The output cell displays the 'Granite Model Refined Response:' followed by a list of three items:

1. Positive: Battery life, Negative: Heat during gaming
2. Negative: Screen brightness outdoors, Positive: Screen colors indoors
3. Positive: Speed, Negative: App crashing

The status bar at the bottom indicates the execution was successful, taking 2 seconds, and completed at 10:36 AM.

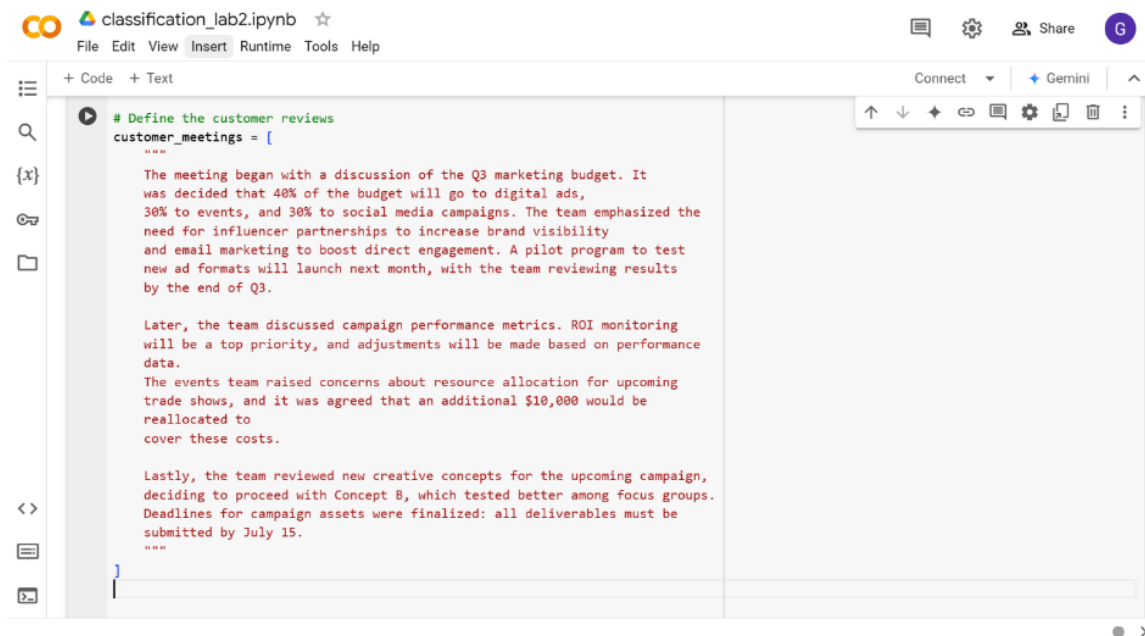
Step 5: Test initial summarization output with default values for parameters**Overview**

In this step, you will input a meeting transcript into the model, run a basic prompt with the default parameters to summarize the meeting, and analyze the output for improvement opportunities.

Instructions

1. Input the meeting transcript simulating a meeting. Type this transcript in the **Instruction** field:

```
# Define the customer reviews
customer_meetings = [
    "The meeting began with a discussion of the Q3 marketing budget. It was decided that 40% of the budget will go to digital ads, 30% to events, and 30% to social media campaigns. The team emphasized the need for influencer partnerships to increase brand visibility and email marketing to boost direct engagement. A pilot program to test new ad formats will launch next month, with the team reviewing results by the end of Q3. Later, the team discussed campaign performance metrics. ROI monitoring will be a top priority, and adjustments will be made based on performance data. The events team raised concerns about resource allocation for upcoming trade shows, and it was agreed that an additional $10,000 would be reallocated to cover these costs. Lastly, the team reviewed new creative concepts for the upcoming campaign, deciding to proceed with Concept B, which tested better among focus groups. Deadlines for campaign assets were finalized: all deliverables must be submitted by July 15."
]
```



The screenshot shows a Jupyter Notebook titled 'classification_lab2.ipynb'. The code cell contains a list of customer meeting reviews. The reviews are as follows:

```
# Define the customer reviews
customer_meetings = [
    """
    The meeting began with a discussion of the Q3 marketing budget. It
    was decided that 40% of the budget will go to digital ads,
    30% to events, and 30% to social media campaigns. The team emphasized the
    need for influencer partnerships to increase brand visibility
    and email marketing to boost direct engagement. A pilot program to test
    new ad formats will launch next month, with the team reviewing results
    by the end of Q3.

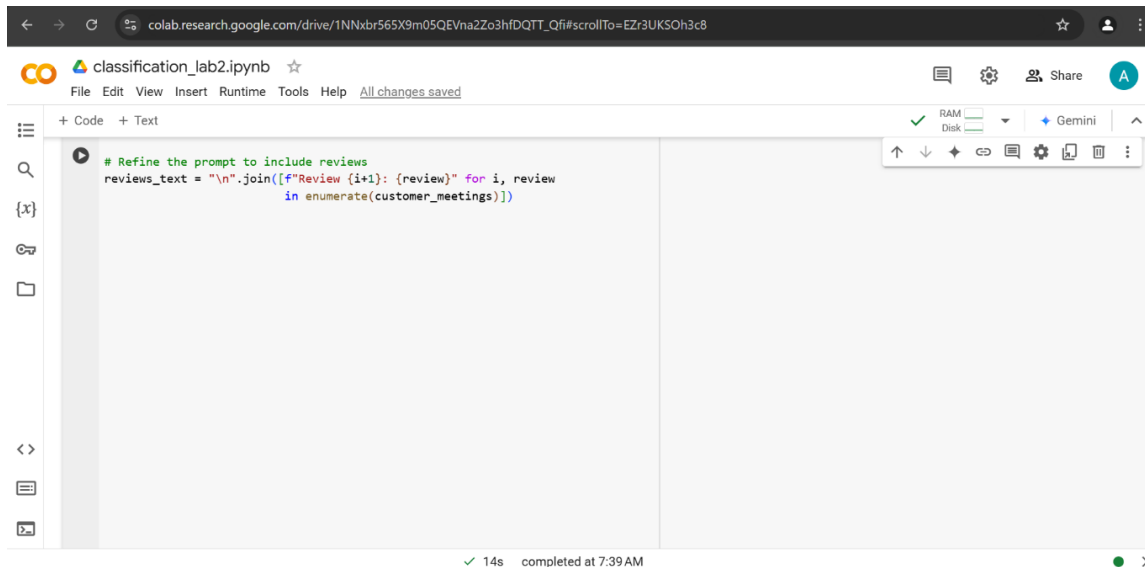
    Later, the team discussed campaign performance metrics. ROI monitoring
    will be a top priority, and adjustments will be made based on performance
    data.

    The events team raised concerns about resource allocation for upcoming
    trade shows, and it was agreed that an additional $10,000 would be
    reallocated to
    cover these costs.

    Lastly, the team reviewed new creative concepts for the upcoming campaign,
    deciding to proceed with Concept B, which tested better among focus groups.
    Deadlines for campaign assets were finalized: all deliverables must be
    submitted by July 15.
    """
]
```

2. Now, input the prompt to include reviews. Type this prompt in the **Instruction** field:

```
# Refine the prompt to include reviews
reviews_text = "\n".join([f"Review {i+1}: {review}" for i, review
in enumerate(customer_meetings)])
```



The screenshot shows the same Jupyter Notebook interface, but the code cell now contains the refined prompt code. The code is as follows:

```
# Refine the prompt to include reviews
reviews_text = "\n".join([f"Review {i+1}: {review}" for i, review
in enumerate(customer_meetings)])
```

At the bottom of the notebook, a status bar indicates '14s completed at 7:39 AM'.

3. Input the default parameters and refined prompt to configure the model's response, focusing on summarizing key points, decisions, and action items from the meeting. Type this code in the **Instruction** field:

```
# Apply default parameters
parameters = {
    "top_k": 0,
    "top_p": 1.0,
    "max_tokens": 256,
    "min_tokens": 0,
    "random_seed": None,
    "repetition_penalty": 1.0,
    "stopping_criteria": "length (256 tokens)",
    "stopping_sequence": None
}

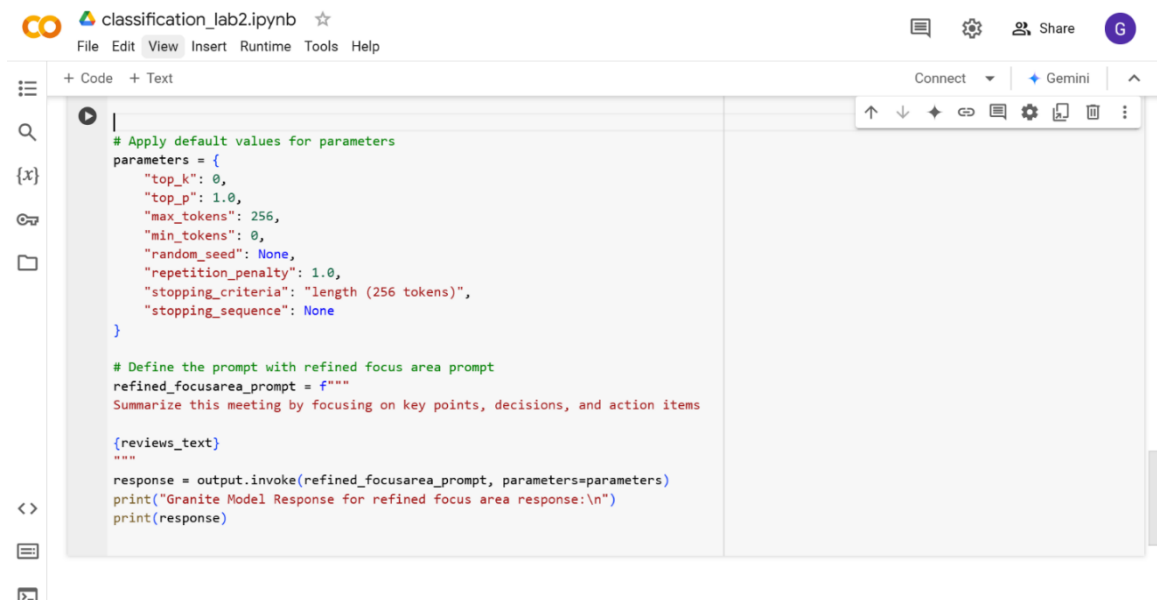
# Refined prompt with example included
refined_focus_prompt = f"""
Summarize this meeting by focusing on key points, decisions, and
action items

{reviews_text}
"""

# Invoke the model with the refined focus prompt
response = output.invoke(refined_focus_prompt,
parameters=parameters)
# Print the response
print("Granite Model Response for refined focus area response:\n")

print(response)
```

Adjust the IBM Granite model parameters to refine the output



The screenshot shows a Jupyter Notebook titled 'classification_lab2.ipynb'. The code in the cell is as follows:

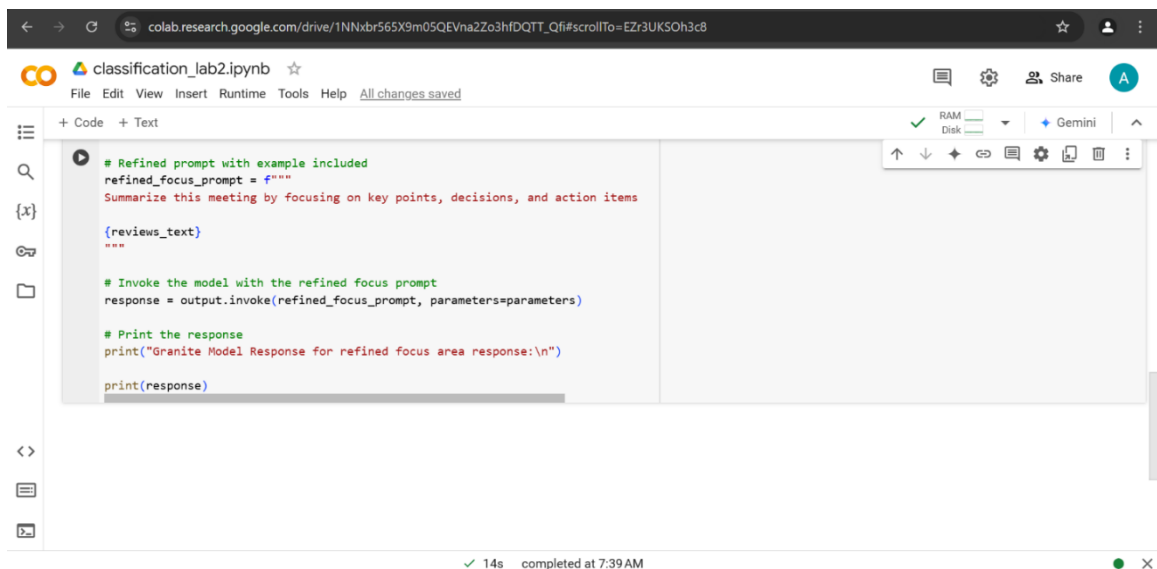
```
# Apply default values for parameters
parameters = {
    "top_k": 0,
    "top_p": 1.0,
    "max_tokens": 256,
    "min_tokens": 0,
    "random_seed": None,
    "repetition_penalty": 1.0,
    "stopping_criteria": "length (256 tokens)",
    "stopping_sequence": None
}

# Define the prompt with refined focus area prompt
refined_focusarea_prompt = f"""
Summarize this meeting by focusing on key points, decisions, and action items

{reviews_text}
"""

response = output.invoke(refined_focusarea_prompt, parameters=parameters)
print("Granite Model Response for refined focus area response:\n")
print(response)
```

4. Select the **Play** button to run the prompt.



The screenshot shows the same Jupyter Notebook after execution. The code is as follows:

```
# Refined prompt with example included
refined_focus_prompt = f"""
Summarize this meeting by focusing on key points, decisions, and action items

{reviews_text}
"""

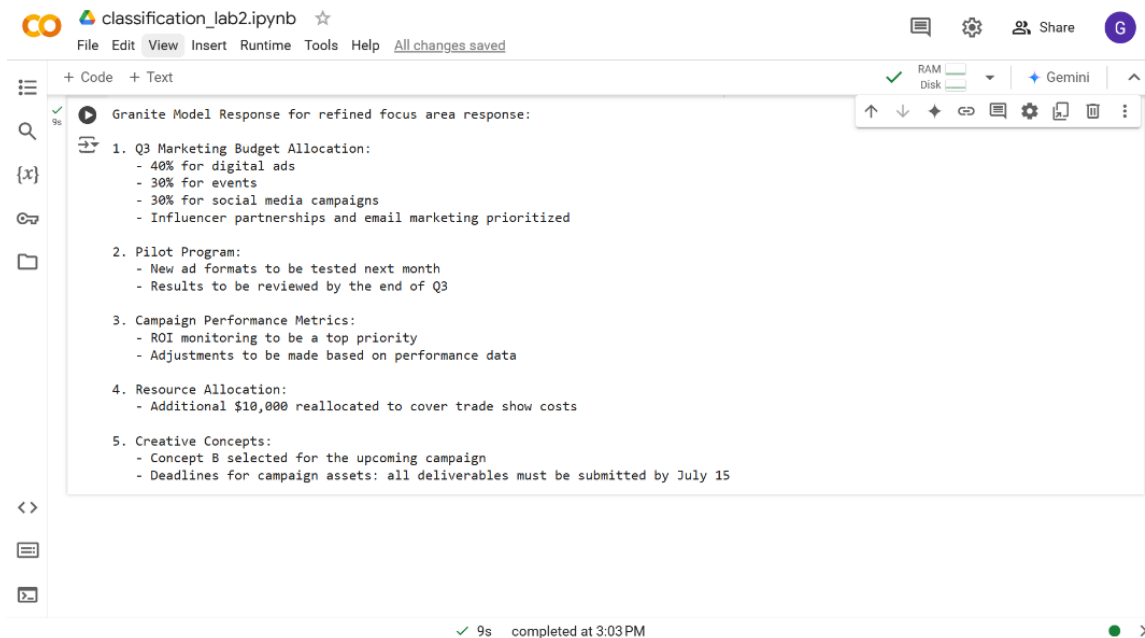
# Invoke the model with the refined focus prompt
response = output.invoke(refined_focus_prompt, parameters=parameters)

# Print the response
print("Granite Model Response for refined focus area response:\n")
print(response)
```

At the bottom of the notebook, a status bar indicates: ✓ 14s completed at 7:39 AM.

5. Read the output that the model generates. Note that the output provides a detailed breakdown of the marketing strategies, but it might be too detailed for scenarios that require more concise summaries. In many contexts, decision makers or stakeholders prefer brief summaries that highlight the key points, saving time while still capturing the essence of the content. A more concise response is more effective, especially for presentations or executive briefs.

You can improve output by adjusting the max_tokens parameter in the next step.



```
classification_lab2.ipynb
File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text
Granite Model Response for refined focus area response:

1. Q3 Marketing Budget Allocation:
   - 40% for digital ads
   - 30% for events
   - 30% for social media campaigns
   - Influencer partnerships and email marketing prioritized

2. Pilot Program:
   - New ad formats to be tested next month
   - Results to be reviewed by the end of Q3

3. Campaign Performance Metrics:
   - ROI monitoring to be a top priority
   - Adjustments to be made based on performance data

4. Resource Allocation:
   - Additional $10,000 reallocated to cover trade show costs

5. Creative Concepts:
   - Concept B selected for the upcoming campaign
   - Deadlines for campaign assets: all deliverables must be submitted by July 15

✓ 9s completed at 3:03 PM
```


Step 6: Adjust a single parameter to improve the summarization output

Overview

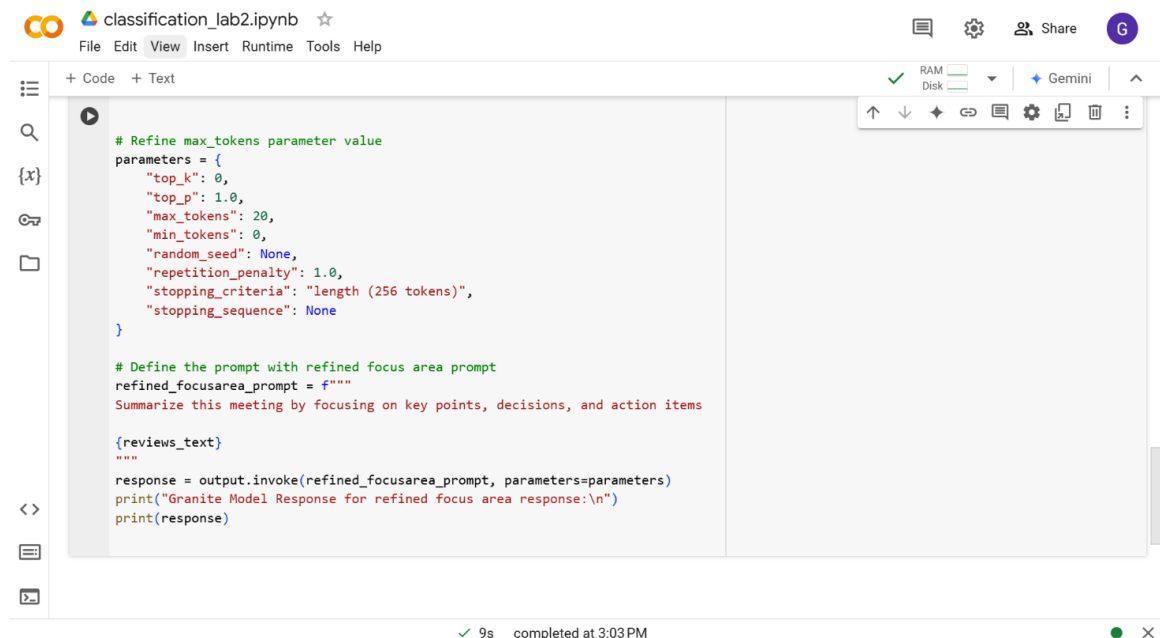
In this step, you will adjust the `max_tokens` parameter to allow the model to generate more concise summaries.

Instructions

1. Update the `max_tokens` parameter by setting it to 20. Type this code in the **Instruction** field:

```
# Refine max_tokens parameter value

parameters = {
    "top_k": 0,
    "top_p": 1.0,
    "max_tokens": 20,
    "min_tokens": 0,
    "random_seed": None,
    "repetition_penalty": 1.0,
    "stopping_criteria": "length (256 tokens)",
    "stopping_sequence": None
}
```



```
classification_lab2.ipynb
File Edit View Insert Runtime Tools Help

+ Code + Text

# Refine max_tokens parameter value
parameters = {
    "top_k": 0,
    "top_p": 1.0,
    "max_tokens": 20,
    "min_tokens": 0,
    "random_seed": None,
    "repetition_penalty": 1.0,
    "stopping_criteria": "length (256 tokens)",
    "stopping_sequence": None
}

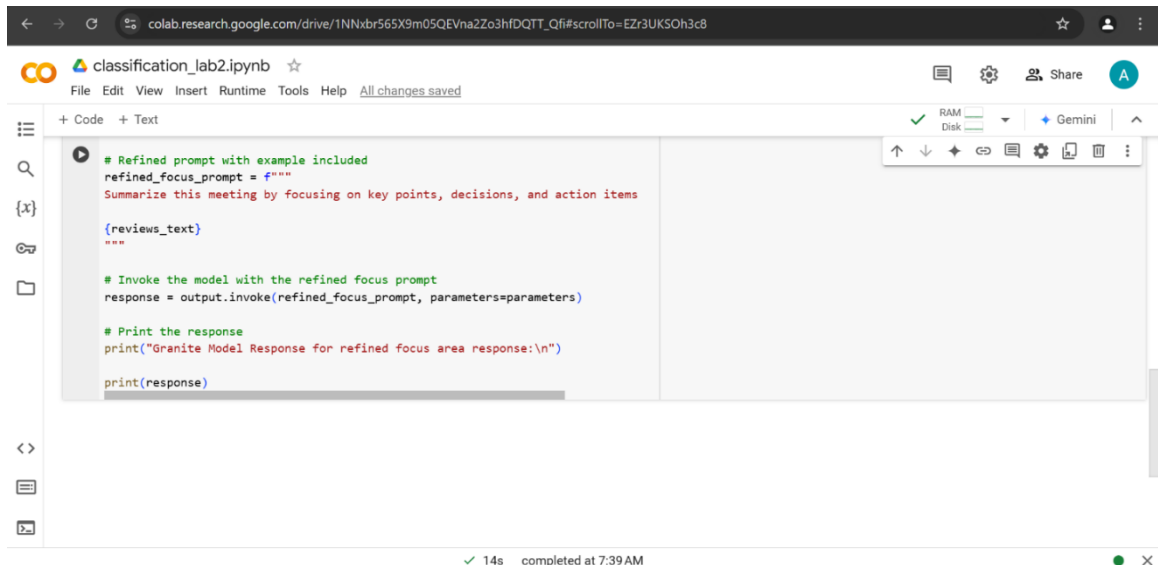
# Define the prompt with refined focus area prompt
refined_focusarea_prompt = f"""
Summarize this meeting by focusing on key points, decisions, and action items

{reviews_text}
"""

response = output.invoke(refined_focusarea_prompt, parameters=parameters)
print("Granite Model Response for refined focus area response:\n")
print(response)

9s completed at 3:03 PM
```

2. Select the **Play** button to run the prompt.



```
# Refined prompt with example included
refined_focus_prompt = """
Summarize this meeting by focusing on key points, decisions, and action items

{reviews_text}
"""

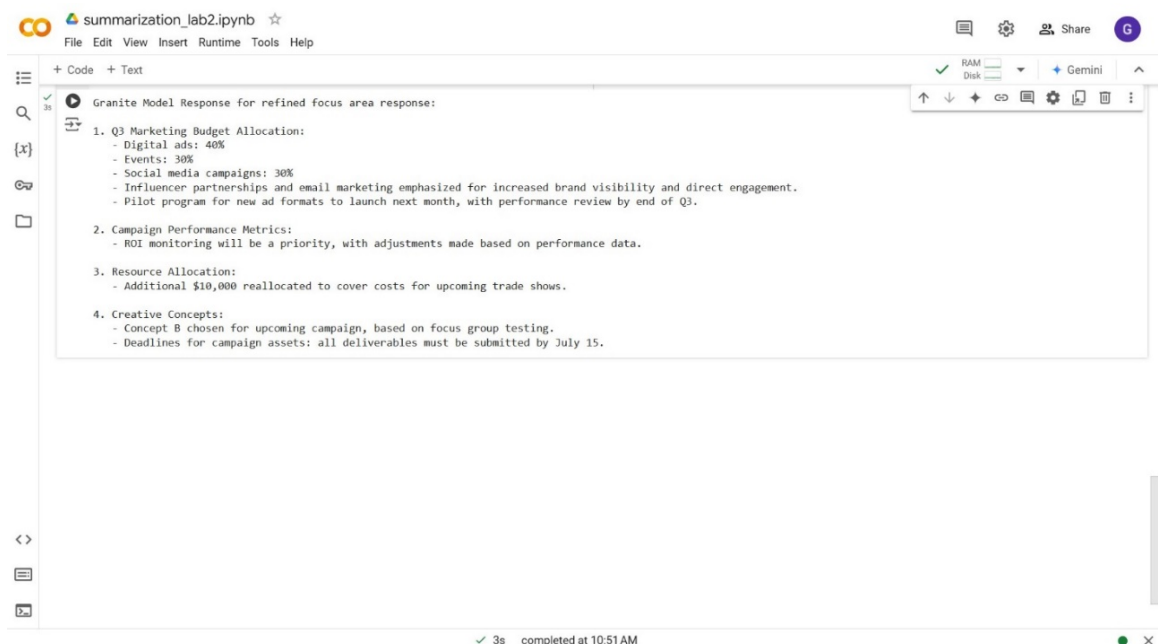
# Invoke the model with the refined focus prompt
response = output.invoke(refined_focus_prompt, parameters=parameters)

# Print the response
print("Granite Model Response for refined focus area response:\n")

print(response)
```

3. Read the output that the model generates. Note that reducing the max_tokens parameter to 20 (max_tokens = 20) significantly condensed the response, prioritizing essential information such as budget allocation, campaign focus, and deadlines. This adjustment distilled key details, eliminating less critical information compared to the default parameter values. However, the output could be further refined by avoiding repetitive words.

You can improve the output by adjusting more parameters in the next step.



```
Granite Model Response for refined focus area response:

1. Q3 Marketing Budget Allocation:
   - Digital ads: 40%
   - Events: 30%
   - Social media campaigns: 30%
   - Influencer partnerships and email marketing emphasized for increased brand visibility and direct engagement.
   - Pilot program for new ad formats to launch next month, with performance review by end of Q3.

2. Campaign Performance Metrics:
   - ROI monitoring will be a priority, with adjustments made based on performance data.

3. Resource Allocation:
   - Additional $10,000 reallocated to cover costs for upcoming trade shows.

4. Creative Concepts:
   - Concept B chosen for upcoming campaign, based on focus group testing.
   - Deadlines for campaign assets: all deliverables must be submitted by July 15.
```

Step 7: Adjust multiple parameters to improve the summarization output

Overview

In this step, you will adjust multiple parameters to generate well-structured summaries.

Instructions

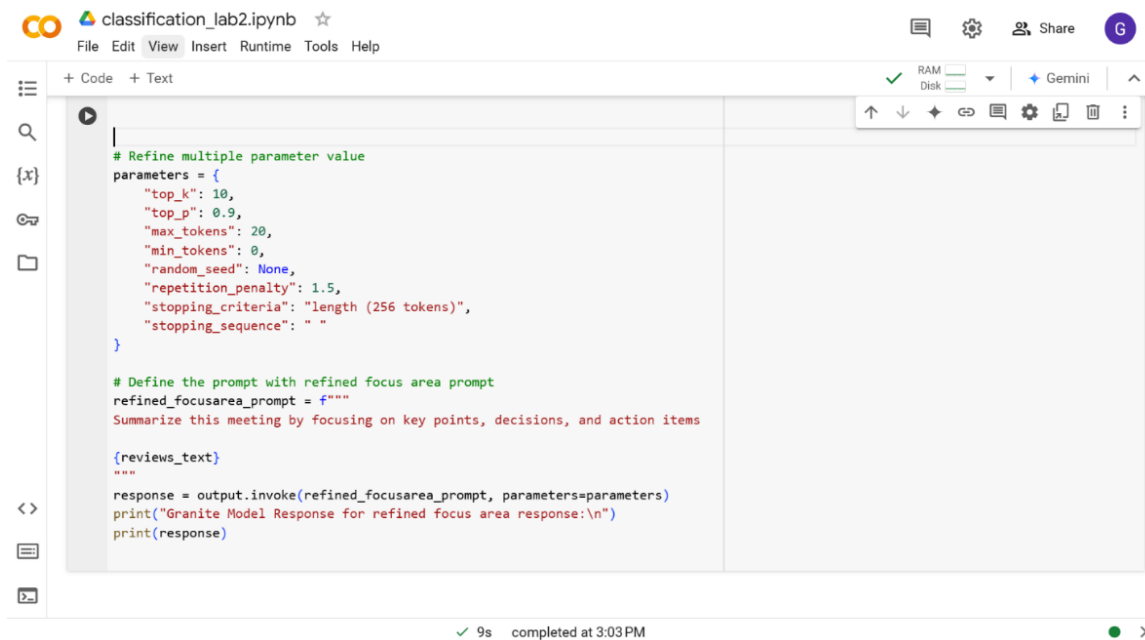
1. Adjust the following parameters to ensure the output quality is enhanced:
 - **top_p**: Maintain at 0.9 for focused, high-probability output.
 - **top_k**: Set to 10 to allow slightly more variation in token selection while maintaining coherence.
 - **max_tokens**: Keep at 20 to ensure concise responses.
 - **repetition_penalty**: Maintain at 1.5 to reduce redundancy.
 - **stopping_sequence**: Keep as a space (" ") to terminate output at natural breaks.

Input the refined values for the required parameters. Type this code in the **Instruction** field:

```
# Refine multiple parameter value

parameters = {
    "top_k": 10,
    "top_p": 0.9,
    "max_tokens": 20,
    "min_tokens": 0,
    "random_seed": None,
    "repetition_penalty": 1.5,
    "stopping_criteria": "length (256 tokens)",
    "stopping_sequence": " "
}
```

Adjust the IBM Granite model parameters to refine the output



The screenshot shows a Jupyter Notebook titled 'classification_lab2.ipynb'. The code defines a set of parameters for the model, including 'top_k', 'top_p', 'max_tokens', 'min_tokens', 'random_seed', 'repetition_penalty', 'stopping_criteria', and 'stopping_sequence'. It then defines a refined focus area prompt and invokes the model using the 'output.invoke' method. The notebook status bar indicates it completed at 3:03 PM.

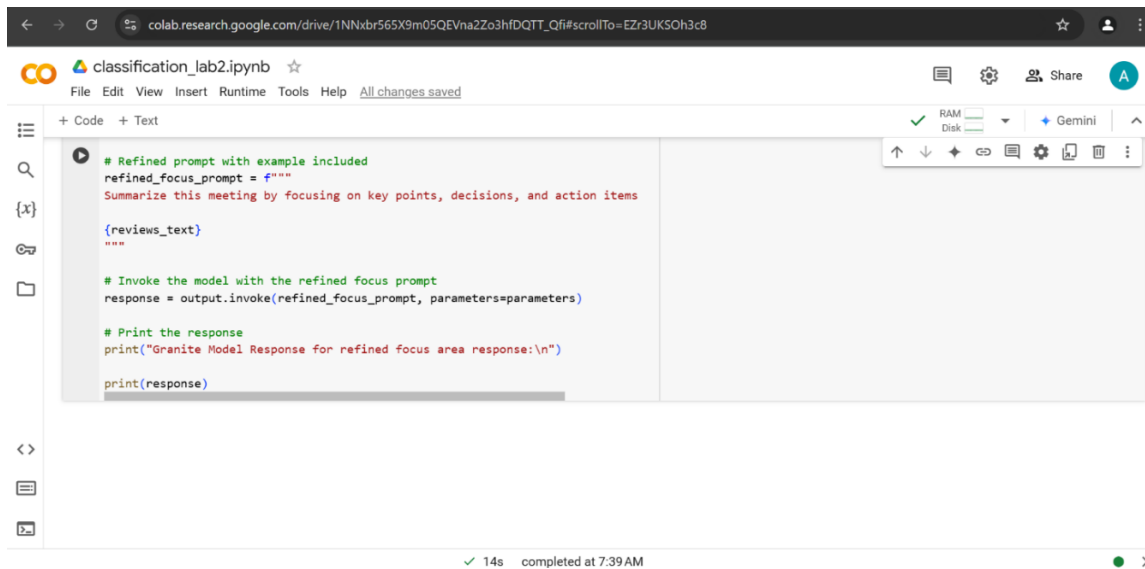
```
# Refine multiple parameter value
parameters = {
    "top_k": 10,
    "top_p": 0.9,
    "max_tokens": 20,
    "min_tokens": 0,
    "random_seed": None,
    "repetition_penalty": 1.5,
    "stopping_criteria": "length (256 tokens)",
    "stopping_sequence": " "
}

# Define the prompt with refined focus area prompt
refined_focusarea_prompt = f"""
Summarize this meeting by focusing on key points, decisions, and action items

{reviews_text}
"""

response = output.invoke(refined_focusarea_prompt, parameters=parameters)
print("Granite Model Response for refined focus area response:\n")
print(response)
```

2. Select the **Play** button to run the prompt.



The screenshot shows the same Jupyter Notebook after running the code. The code now includes the refined focus prompt and the model invocation. The notebook status bar indicates it completed at 7:39 AM.

```
# Refined prompt with example included
refined_focus_prompt = f"""
Summarize this meeting by focusing on key points, decisions, and action items

{reviews_text}
"""

# Invoke the model with the refined focus prompt
response = output.invoke(refined_focus_prompt, parameters=parameters)

# Print the response
print("Granite Model Response for refined focus area response:\n")
print(response)
```

3. Read the output that the model generates. Note that adjusting multiple parameters made the response more concise, avoiding duplicate words. By setting `top_k = 10`, the model selected from the 10 most likely words, controlling vocabulary and structure. Setting `top_p = 0.9` ensured the model chose words that were meaningful and contextually appropriate. Maintaining the `repetition_penalty = 1.5` avoided redundant phrasing, while setting the `stopping_sequence = " "` stopped the response at natural pauses.

The balanced approach towards adjusting multiple model parameters ensure that the final output is accurate and tailored to the stakeholder's needs.

```
summarization_lab2.ipynb
File Edit View Insert Runtime Tools Help All changes saved

+ Code + Text
print(response)

Granite Model Response for refined focus area response:

1. Marketing Budget Allocation: 40% for digital ads, 30% for events, and 30% for social media campaigns.
2. Influencer partnerships and email marketing will be prioritized to increase brand visibility and direct engagement.
3. A pilot program for new ad formats will launch next month, with results reviewed by the end of Q3.
4. ROI monitoring will be a top priority, with adjustments made based on performance data.
5. An additional $10,000 will be reallocated to cover resource allocation for upcoming trade shows.
6. Concept B will be proceeded with for the upcoming campaign, as it tested better among focus groups.
7. All campaign assets must be submitted by July 15.

Colab paid products - Cancel contracts here
3s completed at 10:56 AM
```

Conclusion

In this lab, you used the **granite-3.3-8b-instruct** model and adjusted the model parameters for prompting in two cycles to improve the output quality for classification and summarization tasks. These refinements ensured that output was consistent, precise, and aligned with stakeholder needs, enhancing decision making and productivity.

© Copyright IBM Corporation 2025.

The information contained in these materials is provided for informational purposes only and is provided AS IS without warranty of any kind, express or implied. IBM shall not be responsible for any damages arising out of the use of, or otherwise related to, these materials. Nothing contained in these materials is intended to, nor shall have the effect of, creating any warranties or representations from IBM or its suppliers or licensors, or altering the terms and conditions of the applicable license agreement governing the use of IBM software. References in these materials to IBM products, programs, or services do not imply that they will be available in all countries in which IBM operates. This information is based on current IBM product plans and strategy, which are subject to change by IBM without notice. Product release dates and/or capabilities referenced in these materials may change at any time at IBM's sole discretion based on market opportunities or other factors, and are not intended to be a commitment to future product or feature availability in any way.

IBM, the IBM logo and ibm.com are trademarks of International Business Machines Corp., registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the Web at "Copyright and trademark information" at www.ibm.com/legal/copytrade.shtml.



Please Recycle